

SIMATS ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MOCK INTERVIEW PREPARATION GUIDE

Assessment Tool 3 — Answer Reference Material

Item	Details
Course	Database Management Systems (CSA05)
Course Outcome	CO5 — Query Processing, Optimization & Transaction Management
Bloom's Levels	BL3 (Apply) · BL4 (Analyze)
Activity	Mock Interview (Weightage: 30%)
Total Marks	50 (10 questions × 5 marks each)

Student Declaration

I, KARNAM HARSHITH (Register Number: 192511384), hereby declare that the responses presented in this document represent my own understanding of the subject matter. I have prepared these answers independently for the mock interview assessment and confirm that I have followed all academic integrity guidelines of the institution. I acknowledge that any breach of these rules may lead to disciplinary measures as per university policy.

Signature of the student: _____ Date: _____

This reference material presents each interview question followed by a structured spoken-style answer and a compact illustrative demonstration. The demonstrations are conceptual simulations intended to reinforce the underlying ideas rather than production database output. Additional notes and key takeaways are included after each demonstration to aid revision.

QUESTION 1

Interviewer: “Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?”

Level: BL3 – Apply | Marks: 5

Candidate Response

Query processing in a DBMS typically unfolds in a clear sequence of phases. The first phase is parsing: the system verifies that the submitted SQL statement is syntactically valid and constructs an internal parse tree. Immediately afterward, semantic validation occurs—the catalog is consulted to ensure that every referenced table and column exists and that data types are compatible.

Once the statement is confirmed correct, it is rewritten into a logical plan expressed in relational-algebra operators. At this point the query optimizer becomes active. It enumerates a set of alternative physical plans that differ in join order, choice of join method (nested-loop, hash, merge), and access path (index lookup versus sequential scan). Using catalog statistics—cardinality, index selectivity, and available buffer memory—the optimizer computes an estimated cost, primarily in terms of disk page I/Os and secondarily in CPU operations, for each candidate. The plan with the lowest estimated cost is selected and handed to the execution engine, which materializes the result set and returns it to the client.

Importantly, the optimizer does not choose the plan that appears simplest on paper; it chooses the plan whose projected resource consumption is minimal according to the cost model. Modern optimizers may also apply dynamic programming or genetic algorithms to explore the plan space when the number of joins is large.

Illustrative Demonstration

```
$ python3 query_pipeline_demo.py
```

[1] PARSE & VALIDATE

SQL : SELECT Name, Marks FROM Student WHERE Marks > 80;

Status: syntax OK | tables & columns verified in catalog

[2] CANDIDATE PHYSICAL PLANS

Plan A Full table scan + filter estimated cost $\approx$ 1000 I/Os

Plan B Index seek on Marks + filter estimated cost $\approx$ 45 I/Os

[3] COST-BASED SELECTION

→ Plan B selected (lowest estimated I/O cost)

[4] EXECUTE

Result rows: [('Arun', 88), ('Divya', 91)]

Figure 1.1 — Cost comparison of two candidate plans; the optimizer retains the cheaper index-based strategy.

Key takeaway: Always keep table and index statistics up to date; stale statistics cause the optimizer to choose poor plans even when a good index exists.

QUESTION 2

Interviewer: “What is the difference between heuristic optimization and cost-based optimization? When would you use each?”

Level: BL4 – Analyze | Marks: 5

Candidate Response

Heuristic optimization relies on a collection of fixed transformation rules that are expected to improve performance in almost every situation, independent of the actual data distribution. Classic examples include pushing selection and projection operators as far down the query tree as possible so that subsequent operations process fewer tuples, and eliminating redundant joins or predicates. Because these rules need no statistics, they can be applied quickly and cheaply.

Cost-based optimization, by contrast, generates multiple alternative plans and ranks them by a numeric cost estimate derived from table statistics (row counts, distinct-value counts, index selectivity, buffer-pool size). Decisions such as join order and choice of join algorithm are highly data-dependent; the plan that looks attractive for one size of tables may be disastrous for another. Only a cost model can reliably distinguish them.

In modern systems the two approaches are combined. Heuristic rewrites first reduce the size of intermediate results and shrink the search space; cost-based search is then performed among the remaining candidates for the decisions that truly depend on the data. This hybrid strategy keeps optimization time acceptable even for complex queries.

Illustrative Demonstration

```
$ python3 opt_strategies_demo.py
```

=== HEURISTIC REWRITE (no statistics required) ===

Original tree : JOIN(Student, Enrolment) then SELECT(Marks > 80)

Rule applied : push selection below join

Rewritten : JOIN(SELECT(Student, Marks>80), Enrolment)

Effect : fewer rows enter the join; always beneficial

=== COST-BASED COMPARISON (uses cardinality statistics) ===

Nested-loop join estimated cost = 12000 I/Os

Hash join estimated cost = 420 I/Os

Sort-merge join estimated cost = 680 I/Os

→ Optimizer selects Hash Join (lowest numeric cost)

Guideline: apply heuristics first; use cost model for join order / method.

Figure 2.1 — Heuristic selection push-down together with a cost-based join-method comparison.

Key takeaway: Heuristics are cheap and almost always safe; cost-based decisions are essential whenever relative table sizes can change the best plan.

QUESTION 3

Interviewer: “How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.”

Level: BL3 – Apply | Marks: 5

Candidate Response

In a production application I would never open a fresh TCP connection for every request. Instead I would configure a connection pool that maintains a modest number of already-authenticated connections. Application code borrows a connection from the pool, executes its statements inside an explicit transaction, commits or rolls back, and always returns the connection to the pool—preferably inside a finally block or try-with-resources construct—so that even unexpected exceptions cannot leak connections.

JDBC (Java Database Connectivity) is a pure-Java API. A type-4 driver speaks the database’s native network protocol directly; the only requirement is that the driver JAR be present on the classpath. No operating-system configuration is needed. ODBC (Open Database Connectivity) is a language-neutral C-level API. It routes calls through an OS-registered Driver Manager and a Data Source Name (DSN). This design lets many host languages share the same underlying driver, but each client machine must have the DSN correctly configured.

In practice most Java services use a pool such as HikariCP or Tomcat JDBC on top of a JDBC driver, while polyglot environments or legacy C/C++ components may still rely on ODBC.

Illustrative Demonstration

```
$ python3 connectivity_compare.py
```

```
--- JDBC (Java-specific, pure-Java driver) ---
Class.forName("com.mysql.cj.jdbc.Driver");      // load driver
Connection c = DriverManager.getConnection(url,u,p); // direct TCP
Statement s = c.createStatement();
ResultSet r = s.executeQuery("SELECT * FROM Student");
while (r.next()) { /* process row */ }
c.close();
```

```
--- ODBC (language-neutral, Driver Manager + DSN) ---
SQLAllocHandle(SQL_HANDLE_ENV, ...);
SQLConnect(hdbc, "MyDSN", ..., user, pw); // via configured DSN
SQLExecDirect(hstmt, "SELECT * FROM Student", ...);
SQLFetch(hstmt); // row-by-row
SQLDisconnect(hdbc);
```

Key contrast: JDBC needs only a JAR; ODBC needs a system DSN.

Figure 3.1 — Direct JDBC driver path versus ODBC Driver-Manager / DSN path.

Key takeaway: Always use a connection pool in production; never create a new physical connection per request.

QUESTION 4

Interviewer: “What are ACID properties? Give a real-world example where each property is critical.”

Level: BL3 – Apply | Marks: 5

Candidate Response

Atomicity guarantees that a transaction is an indivisible unit: either every change it makes becomes permanent or none of them do. A classic illustration is a funds transfer between two bank accounts. If the system fails after debiting the source but before crediting the destination, the entire transfer must be undone so that money does not disappear.

Consistency requires that a transaction take the database from one valid state to another, respecting all declared constraints. In an airline reservation system, two passengers must never be confirmed for the same physical seat; the transaction logic and integrity constraints together preserve this invariant.

Isolation ensures that concurrent transactions do not observe one another's intermediate, uncommitted results. When two customers attempt to purchase the last remaining concert ticket at the same moment, each must not both succeed merely because each saw a stale "available" count.

Durability promises that once a transaction has committed, its effects survive subsequent failures. After an ATM has dispensed cash and the withdrawal has been acknowledged, the corresponding debit must remain recorded even if the server crashes a moment later.

Together the four properties give applications a reliable programming model: the programmer can reason about each transaction as if it executed alone and completed successfully, even though the system may interleave many transactions and may crash at any time.

Illustrative Demonstration

```
$ python3 acid_bank_demo.py
```

```
Initial balances : { 'A': 1000, 'B': 500 }
```

```
--- Scenario 1: successful transfer (all ACID properties hold) ---
```

```
BEGIN transfer 200 from A to B
```

```
debit A → 800
```

```
credit B → 700
```

```
COMMIT (changes made durable)
```

```
Balances after : { 'A': 800, 'B': 700 } total still 1500
```

```
--- Scenario 2: crash after debit, before credit ---
```

```
BEGIN transfer 300 from A to B
```

```
debit A → 500
```

```
** simulated failure **
```

```
ROLLBACK → balances restored to { 'A': 800, 'B': 700 }
```

```
(Atomicity preserved; no partial update remains)
```

Figure 4.1 — Successful commit versus mid-transfer failure rolled back atomically.

Key takeaway: Atomicity and durability are enforced by the recovery manager (logging); isolation is enforced by the concurrency-control manager (locks or MVCC).

QUESTION 5

Interviewer: "Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?"

Level: BL4 – Analyze | Marks: 5

Candidate Response

A dirty read occurs when one transaction reads a value written by another transaction that has not yet committed. If the writer later aborts, the reader has acted upon data that never officially existed.

A lost update arises when two transactions both read the same item, each computes a new value, and the later write silently overwrites the earlier one, erasing that update completely.

An unrepeatable read is observed when a transaction reads the same row twice and obtains two different values because a concurrent transaction committed a modification between the two reads.

Two-Phase Locking (2PL) eliminates all three anomalies by requiring a transaction to acquire the appropriate lock before accessing an item and to release locks only after it has finished acquiring new ones (the shrinking phase). An exclusive lock held by a writer prevents any other transaction from reading or writing that item until the writer commits, thereby blocking dirty reads and lost updates. Holding shared locks until the shrinking phase likewise prevents another transaction from altering a value that a still-active reader has already observed, solving unrepeatable reads.

Additional anomalies such as the phantom read (a range query returning different sets of rows on successive executions) are prevented by next-key or gap locking, which many commercial engines implement as an extension of 2PL.

Illustrative Demonstration

```
$ python3 concurrency_anomalies.py
```

WITHOUT LOCKING (lost-update anomaly)

Initial balance = 100

T1 reads 100

T2 reads 100 ← interleaved before T1 writes

T1 writes 150

T2 writes 130 ← T1's +50 update is lost

Final balance = 130 (should have been 180)

WITH BASIC 2PL (lost update prevented)

T1 requests X-lock on balance → GRANTED

T1 reads 100, writes 150, COMMITS, releases lock

T2 requests X-lock → waits, then obtains lock

T2 reads 150, writes 180, COMMITS

Final balance = 180 (both updates preserved)

Figure 5.1 — Lost-update anomaly without locking, then the same interleaving under 2PL.

Key takeaway: 2PL guarantees conflict serializability; stricter variants (Strict / Rigorous) also eliminate cascading aborts.

QUESTION 6

Interviewer: “What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.”

Level: BL4 – Analyze | Marks: 5

Candidate Response

A deadlock is a circular wait: each transaction in a set is waiting for a lock that is held by the next transaction in the cycle, so none of them can ever progress. Detection is performed by maintaining a Wait-For Graph in which an edge $T_i \rightarrow T_j$ means that T_i is waiting for a lock currently held by T_j . Periodically a depth-first search is run over the graph; the presence of a back-edge to a node that is still on the recursion stack signals a cycle. The system then selects one transaction from the cycle as a victim, aborts it, and releases its locks, thereby breaking the deadlock.

Two well-known prevention techniques use transaction timestamps. Wait-Die allows an older transaction to wait for a younger one, but forces a younger transaction that requests a lock held by an older transaction to abort (“die”) and restart later with the same timestamp. Wound-Wait reverses the roles: an older transaction “wounds” (pre-empt) a younger lock holder. A simpler alternative that does not require a graph is timeout-based prevention: if a transaction has been waiting longer than a configured threshold, the system assumes a deadlock and aborts it.

Victim selection policies may favour the transaction that has done the least work, the youngest transaction, or the one holding the fewest locks, in order to minimise the cost of rollback and restart.

Illustrative Demonstration

```
$ python3 deadlock_demo.py
```

=== DETECTION via Wait-For Graph ===

Edges: $T1 \rightarrow T2$, $T2 \rightarrow T3$, $T3 \rightarrow T1$

DFS discovers cycle $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$

Action: abort victim (e.g. $T1$) and release its locks

=== PREVENTION 1: Wait-Die (timestamp based) ===

$T1(ts=5)$ requests lock held by $T2(ts=10)$

→ requester is older → $T1$ waits

$T2(ts=10)$ requests lock held by $T1(ts=5)$

→ requester is younger → $T2$ dies (aborts & restarts)

=== PREVENTION 2: Timeout ===

$T1$ has waited 12 s > threshold 5 s → abort $T1$ and retry

Figure 6.1 — Cycle detection in the Wait-For Graph together with Wait-Die and timeout prevention.

Key takeaway: Detection is reactive and needs a Wait-For Graph; prevention (Wait-Die / Wound-Wait / timeout) avoids cycles before they form.

QUESTION 7

Interviewer: “Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?”

Level: BL4 – Analyze | Marks: 5

Candidate Response

Under the Immediate-Update policy a transaction’s modifications may be written to disk as soon as they occur, even before the transaction commits. Consequently, recovery after a crash must perform both a REDO pass (to re-apply the effects of committed transactions whose pages may not have reached stable storage) and an UNDO pass (to reverse the effects of uncommitted transactions whose pages may already have been written).

Deferred Update, conversely, keeps every change of a transaction in a private buffer and flushes those pages to disk only after the transaction has committed. Uncommitted writes therefore never appear on disk; recovery needs only a REDO pass for the committed transactions, and an UNDO phase is unnecessary by design.

Deferred Update is attractive when transactions are short-lived and their write sets fit comfortably in memory, because recovery logic is simpler. Immediate Update is preferred for long-running transactions that produce large volumes of modified pages; buffering everything until commit would consume excessive memory. The extra complexity of maintaining before-images for UNDO is accepted in exchange for better scalability. Most commercial systems (for example ARIES-based recovery) follow a form of immediate update with steal/no-force buffer policies.

Illustrative Demonstration

```
$ python3 recovery_policy_demo.py
```

IMMEDIATE UPDATE (writes may reach disk before commit)

REDO pass (committed txns) : re-apply $T1 \rightarrow X = 50$

UNDO pass (uncommitted) : restore $T2 \rightarrow Y = 20$

(both REDO and UNDO required)

DEFERRED UPDATE (writes flushed only after commit)

REDO pass (committed only) : re-apply $T1 \rightarrow X = 50$

Uncommitted $T2$ writes never reached disk → no UNDO needed

Preference summary:

Short / small write-set → Deferred (simpler recovery)

Long / large write-set → Immediate (memory-friendly)

Figure 7.1 — Same logical log recovered under Immediate Update (REDO+UNDO) versus Deferred Update (REDO only).

Key takeaway: Immediate update needs both REDO and UNDO; deferred update needs only REDO but may consume more memory for long transactions.

QUESTION 8

Interviewer: “Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?”

Level: BL3 – Apply | Marks: 5

Candidate Response

Shadow paging never overwrites a page in place. When a transaction begins, it obtains a private copy of the page table (the current page table). Any page the transaction modifies is written to a newly allocated disk frame; only the pointer inside the current page table is updated. The original “shadow” page table continues to point at the untouched frames. Commit is performed by a single atomic action: the current page table is flushed to stable storage and the system’s global pointer is switched from the old shadow table to the new one.

The principal advantage is that no write-ahead log is required. Recovery after a crash that occurs before commit consists simply of discarding the unreferenced current page table; the shadow table already describes a consistent database. The main limitations are fragmentation (updated pages become scattered across the disk, harming locality), the need for garbage collection of orphaned frames, and weak support for fine-grained concurrency. Log-based schemes such as ARIES remain far more common in commercial systems precisely because they integrate cleanly with record-level locking and concurrent transactions.

Shadow paging can still be attractive for single-user or embedded systems where simplicity of recovery outweighs the drawbacks of fragmentation and limited concurrency.

Illustrative Demonstration

```
$ python3 shadow_paging_demo.py
```

Shadow page table (committed state):

```
{0: 'A_v1', 1: 'B_v1', 2: 'C_v1'}
```

Transaction begins → private current table = copy of shadow

Write page 1 → allocate new frame 'B_v2_NEW'

```
Current table now: {0:'A_v1', 1:'B_v2_NEW', 2:'C_v1'}
```

Shadow table unchanged

--- Crash before commit ---

Discard current table; keep shadow → database consistent

--- Successful commit ---

Flush current table, atomically swap shadow pointer

```
New shadow: {0:'A_v1', 1:'B_v2_NEW', 2:'C_v1'}
```

Figure 8.1 — Copy-on-write page allocation, atomic commit swap, and pre-commit crash recovery.

Key takeaway: Shadow paging gives instant recovery without a log, but at the cost of fragmentation and poorer concurrency support.

QUESTION 9

Interviewer: “What is a serializability schedule? How do you test for conflict serializability using a precedence graph?”

Level: BL4 – Analyze | Marks: 5

Candidate Response

A schedule is said to be conflict-serializable when the relative order of all conflicting operations is identical to the order that would appear in some serial (one-transaction-at-a-time) execution of the same set of transactions. In other words, concurrent execution is safe because its observable effect matches that of a valid serial order.

To test conflict serializability one constructs a precedence (or serialization) graph: a node is created for every transaction, and a directed edge $T_i \rightarrow T_j$ is drawn whenever an operation of T_i conflicts with and precedes an operation of T_j on the same data item. Two operations conflict when they access the same item and at least one of them is a write. The schedule is conflict-serializable if and only if the resulting graph is acyclic. A depth-first search detects cycles in linear time; when the graph is acyclic any topological ordering yields an equivalent serial schedule.

View serializability is a weaker (more permissive) notion that considers only the final reads and writes visible to each transaction; it is NP-complete to test in general, which is why practical systems rely on conflict serializability and locking protocols that guarantee it.

Illustrative Demonstration

```
$ python3 serializability_check.py
```

Schedule S1: R1(A) W2(A) R2(B) W1(B)

Conflicting pairs $\rightarrow$ edges: $T_1 \rightarrow T_2$ (via A), $T_2 \rightarrow T_1$ (via B)

Cycle check (DFS): CYCLE FOUND

$\Rightarrow$ S1 is NOT conflict-serializable

--- Contrast: a serializable schedule ---

Schedule S2: R1(A) W1(A) R2(A) W2(A) R1(B) W1(B)

Precedence edges: $[T_1 \rightarrow T_2]$

Cycle check: no cycle $\rightarrow$ serializable as T_1 then T_2

Figure 9.1 — A cyclic (non-serializable) schedule contrasted with an acyclic (serializable) schedule.

Key takeaway: Acyclic precedence graph $\Leftrightarrow$ conflict serializable; any topological order is a valid equivalent serial order.

QUESTION 10

Interviewer: “Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?”

Level: BL4 – Analyze | Marks: 5

Candidate Response

Basic 2PL permits a transaction to release locks as soon as the shrinking phase begins, which may be before the transaction itself commits. Serializability is still guaranteed, yet cascading aborts become possible: if a second transaction reads a value whose exclusive lock has already been released by a first transaction, and that first transaction later aborts, the second transaction must also be aborted.

Strict 2PL retains all exclusive (write) locks until the transaction commits or aborts; shared (read) locks may still be released earlier. Because no other transaction can ever observe an uncommitted write, cascading aborts are eliminated.

Rigorous 2PL holds every lock—shared or exclusive—until commit or abort. The resulting behaviour is especially easy to reason about, because transactions appear to execute in the order of their commit points.

In practice Strict 2PL is the dominant choice. It delivers the most important practical benefit—absence of cascading aborts—while still allowing shared locks to be released a little earlier than Rigorous 2PL would permit, thereby improving concurrency. Commercial engines such as MySQL InnoDB and Oracle implement Strict 2PL as their default locking discipline for the repeatable-read isolation level.

Many systems further strengthen isolation with multi-version concurrency control (MVCC) so that readers do not block writers; the underlying write locks, however, still follow a strict or rigorous discipline.

Illustrative Demonstration

```
$ python3 two_phase_variants.py
```


BASIC 2PL

T1: lock A, write A, unlock A ← released before commit

T1: lock B, write B, commit

Risk: another txn may read A's uncommitted value → cascade on abort

STRICT 2PL

T1: lock A (X), write A

T1: lock B (X), write B

T1: commit → both X-locks released only now

Benefit: no other txn can see uncommitted writes

RIGOROUS 2PL

All locks (S or X) held until commit/abort

Benefit: simplest reasoning; commit-order appearance

Most common in production: STRICT 2PL

(no cascading aborts + modest extra concurrency vs. Rigorous)

Figure 10.1 — Lock-release timing compared across Basic, Strict and Rigorous 2PL.

Key takeaway: Strict 2PL is the practical default—serializability without cascading aborts, plus slightly better concurrency than Rigorous 2PL.

— *End of Mock Interview Answer Reference* —